Prerequisites

1. Euclidean Distance

Euclidean distance is the straight-line distance between two points in a Euclidean space (also known as the L2 norm). In Machine Learning (ML), points represent **data samples**, and axes represent **features** (e.g., age, income, house size).

Where Euclidean Distance is Used in ML

Many machine learning algorithms rely on measuring "similarity" or "closeness" between data points:

Algorithm Type	Role of Euclidean Distance
k-Nearest Neighbors (k-NN)	Finds the k closest training points to classify a new query point based on shortest distance.
K-Means Clustering	Assigns data points to the nearest cluster centroid and recalculates cluster centers.
Hierarchical Clustering	Computes proximity matrices to merge similar clusters.
Recommendation Systems	Identifies users or items with similar feature vectors (collaborative filtering).
Dimensionality Reduction	Algorithms like t-SNE or UMAP preserve pairwise Euclidean distances in lower dimensions.

2. Manhattan distance

Manhattan distance (also known as City Block distance, Taxicab distance, or the **L1 norm**) measures the distance between two points by summing the absolute differences of their Cartesian coordinates.
Instead of measuring the straight-line distance, it calculates the path you would take if you were walking along a strictly grid-based city street layout—moving only horizontally and vertically.

Manhattan Distance vs. Euclidean Distance

Metric	Manhattan Distance (L1)	Euclidean Distance (L2)
Path Geometry	Grid/taxicab steps (right angles)	Straight-line displacement
Formula	$\sum \lvert pi - qi \rvert$	$\text{sqrt}(\sum (pi - qi)2)$
Outlier Sensitivity	**Less sensitive** (linear penalty)	**Highly sensitive** (squares large deviations)
High Dimensions	Performs better as dimensions grow	Suffers heavily from the Curse of Dimensionality

Key Applications in Machine Learning

1. **High-Dimensional Data:** Preferred over Euclidean distance when dealing with sparse or high-dimensional feature spaces (e.g., text categorisation, TF-IDF vectors, gene data).
2. **Robustness to Outliers:** Useful in algorithms like k-NN or K-Means when the dataset contains extreme noise or outliers, because it doesn't square error terms.

3. **Lasso Regression (L1 Regularisation):** Adds the absolute sum of weights as a penalty term to force unimportant feature coefficients to zero (feature selection).

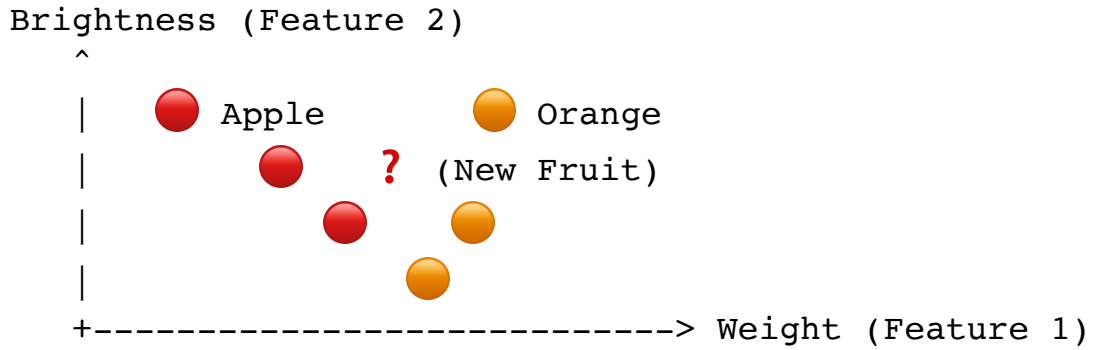
4. **Grid-based Movement & Pathfinding:** Widely used in game AI (A* search algorithm) for board games, mazes, and grid maps where diagonal movement is restricted.

3. kNN (k Nearest Neighbours)

k-Nearest Neighbours (k-NN) is a simple, intuitive machine learning algorithm based on a single core principle: **"Birds of a feather flock together."**
If you want to know what category a new object belongs to—or predict a value for it—you simply look at its closest neighbours in the dataset and let them vote.

How It Works in 4 Steps-

Imagine you are standing in a fruit market with a mystery fruit, and you want to guess if it's an **Apple** or an **Orange** based on its **weight** and **colour brightness**.



1. **Plot the Data:** Place all existing data points on a graph where each axis represents a feature (e.g., Weight on the x-axis, Brightness on the y-axis).

2. **Choose** k
   k: Select how many neighbours (k) get to vote. (e.g., k=3)

3. **Calculate Distances:** Find the k training points physically closest to the new point using a distance metric (usually Euclidean distance or Manhattan distance).

4. Take a Majority Vote:

- **Classification:** If 2 out of 3 neighbours are Apples, the new fruit is classified as an **Apple**.
- **Regression:** If predicting a number (like price), calculate the **average** price of those 3 neighbours.

Choosing the Right k

The value of k controls how sensitive your model is to noise:

Value of k	Effect	Potential Issue
k=1 (Too Small)	Considers only the single closest point.	**Overfitting:** Extremely sensitive to noise and outliers.
k=15(Balanced)	Smooths out noise by averaging across a broader neighborhood.	**Generalization:** Captures underlying trends well.
k=100 (Too Large)	Includes distant, irrelevant points.	**Underfitting:** The majority class in the whole dataset always wins.

Pro-Tip: For binary classification (two classes), always choose an odd number for k (e.g.,k=3, 5, 7) to prevent tie votes!

Key Pros & Cons
Pros

- **Zero Training Phase ("Lazy Learner"):** It doesn't learn a complex mathematical formula; it simply stores the data and computes distances when predicting.
- **Intuitive & Interpretable:** Easy to explain why a decision was made by looking at the neighbours.
- **Non-Linear:** Works well when decision boundaries are complex curves.

Cons

- **Slow Inference:** Calculating distances to *every single training point* for new data gets very slow with massive datasets.
- **Requires Feature Scaling:** Features measured in large numbers (e.g., Salary in thousands) will completely dominate smaller features (e.g., Age in years).
- **Curse of Dimensionality:** As features grow into hundreds of dimensions, distance metrics lose their ability to distinguish "near" from "far."